

CUESTIONES Y EJERCICIOS

1. Vamos a rescatar la clase Vehiculo que hicimos en el tema 1. Está en el repositorio de esta unidad didáctica con el nombre: U1.Extra.Vehiculo.zip. A continuación, vamos a crear una clase Persona que pueda ser propietario de un vehículo, atendiendo al siguiente modelo:

Persona
- nombre: texto - dni: texto - coche: Vehiculo
+ Persona () + Persona (nombre: texto, dni: texto) + comprarCoche(coche: Vehículo) + venderCoche() + arrancarCoche() + apagarCoche() + recorrerDistancia(numKm: real) + reponerCombustible(numlitros: real) + getters de todas las propiedades + setters solo de <i>nombre</i> y <i>dni</i>

Además, debes tener en cuenta que:

- El método *comprarCoche* asigna el parámetro de entrada sobre la propiedad coche.
- El método *venderCoche* pone la propiedad coche a nulo.
- El método *arrancarCoche* y *apagarCoche* utilizan los métodos correspondientes del objeto Vehículo asociado a la persona.
- El método *recorrerDistancia* utiliza el método correspondiente del objeto Vehículo asociado pasándole a su vez el número de kilómetros recorridos a este método.
- El método *reponerCombustible* utiliza el método correspondiente del objeto Vehículo asociado pasándole a su vez el número de litros a reponer a este método.

En el caso de que la persona no posea ningún coche y se intente usar alguno de sus métodos o se intenta vender, se debe emitir un mensaje de “Error: la persona no tiene coche”

Por último, debe crear una clase PruebaPersonaCoche que pruebe todos los métodos de Persona.

2. Vamos a crear dos clases que se relacionen mediante composición y que “se asocien mutuamente”, atendiendo al siguiente modelo:

Cliente	Usuario
- nombre: texto	- nombre: texto
- dni: texto	- password: texto
- direccionPostal: texto	- cliente: Cliente
- email: texto	+ Usuario (nombre: texto, pass: texto, Cliente cli)
- usuario: Usuario	+ getters y setters de todas las propiedades
+ Cliente (nombre: texto, dni: texto, Usuario usu)	
+ Cliente (nombre: texto, dni: texto)	
+ getters y setters de todas las propiedades	

Si observas los constructores, queremos que nuestro modelo permita crear un cliente sin un usuario asociado, pero NO permita crear un usuario sin cliente asociado. Para ello añadiremos la siguiente lógica a los constructores:

- Si se intenta crear un usuario y nos pasan un *null* como valor para la propiedad cliente entonces se debe emitir el mensaje “Error: este usuario NO tiene un cliente asociado” y además se modificarán las propiedades nombre y password para que contengan el texto “error”
- Si se intenta crear un cliente y nos pasan un *null* como valor para la propiedad usuario entonces se creará el cliente con normalidad, almacenando un *null* en la propiedad usuario, pero se mostrará el mensaje “Advertencia: cliente creado sin usuario asociado”.

Crea una clase de pruebas con un método *main* que realice:

- Crea un cliente sin usuario llamado “Pepe”, con el DNI que quieras.
- Crea un usuario de nombre “pepeuser” con la password que quieras asociado al cliente “Pepe”.
- Crea un usuario “erroruser”, con la contraseña que quieras que no se asocie a ningún cliente.
- Imprime el nombre y password del usuario que acabas de crear.
- Crea un cliente sin usuario llamado “Juan”, con el DNI que quieras.
- Crea un usuario de nombre “juanuser” con la password que quieras asociado al cliente “Juan”.
- Ahora utiliza el método *setUsuario(...)* del cliente “Juan” para asociarlo con el usuario “juanuser”.
- Pídele al cliente “Juan” que te devuelva su usuario asociado y después cámbiale el valor de la contraseña.

3. Vamos a crear tres clases que utilicen la composición para relacionarse. Para ello simularemos que tenemos un registro de las ventas de coches de un concesionario, atendiendo al siguiente modelo:

Cliente	Coche
- nombre: texto - dni: texto - direccionPostal: texto - email: texto	- marca: texto - modelo: texto - matricula: texto
+ Cliente (nombre: texto, dni: texto) + getters y setters de todas las propiedades	+ Coche (marca: texto, modelo: texto, matricula: texto) + getters y setters de todas las propiedades

Venta
- fecha: texto - importe: real - cliente: Cliente - coche: Coche
+ constructor con todos los parámetros + getters y setters de todas las propiedades

Crea una clase de pruebas con un método *main* que realice las siguientes acciones:

- Crea tres clientes llamados “John”, “Anne” y “Tom”, con el DNI que quieras.
- Ahora crea los objetos necesarios para reflejar la venta de un coche Seat Ibiza con matrícula “1234-IBZ” comprado por John en el día de ayer por 12.000€.
- Ahora crea los objetos necesarios para reflejar la venta de un coche Opel Corsa con matrícula “4321-COR” comprado por Anne en el día de hoy por 9.900€.
- Ahora crea los objetos necesarios para reflejar la venta de un coche Seat Toledo con matrícula “5678-TOL” comprado por Tom antes de ayer por 12.900€.

Piensa en toda la información que tendrías disponible si tuviéramos un array de objetos Venta que almacenase todas las ventas que ha realizado el concesionario en un año.

Si nuestras clases fueran entidades de una BD ¿cuál sería la cardinalidad de las relaciones entre ellas (1:1, 1:N, N:M)?

4. Crea las siguientes clases:

Moneda	Dinero
- nombre: texto	- cantidad: real
- simbolo: texto	- moneda: Moneda
+ constructor con todos los parámetros	+ constructor con todos los parámetros
+ getters todas las propiedades	+ getters todas las propiedades
	+ toString() devuelve texto

La clase Moneda representará la moneda de un país y la clase Dinero representará una cantidad de moneda. Observa el siguiente ejemplo de cómo podrían crearse objetos de ambas clases:

```
Moneda euro = new Moneda("euro", "€");
Dinero precioCamiseta = new Dinero(12.99, euro);
```

También podríamos hacerlo con una notación más abreviada:

```
Dinero precioTablet = new Dinero(125.99, new Moneda("euro", "€"));
```

La cantidad de la clase Dinero siempre deberá ser positiva, en el caso de que se quiera crear un objeto Dinero con una cantidad negativa, se le cambiará el signo para que se guarde siempre un número positivo. Si se da esta situación se debe imprimir un mensaje de advertencia en la consola.

El método toString() de la clase Dinero debe devolver una cadena de texto que se corresponda con la concatenación de la cantidad y el símbolo de la moneda. Observa:

```
System.out.println("Precio: "+precioTablet.toString());
```

Se imprime:

Precio: 125.99€

Crea una clase de pruebas con un método *main* que realice las siguientes acciones:

- Crea dos objetos moneda uno para el "euro" y otro para el "dólar"
- Después crea dos objetos dinero con los valores 300.50€ y -128.99\$
- Imprime los objetos en pantalla.
- Crea otro objeto con la notación más abreviada que se corresponda con 12.78Fr (francos suizos).

Nota de interés: este tipo de objetos se les llama "value object" y se usan en la industria como objetos "inmutables", es decir que no se les puede modificar una vez creados (observa que no tienen setters)

5. Continuando con el ejercicio anterior, añade una nueva clase:

Articulo
- código: entero - nombre: texto - precio: Dinero
+ constructor con todos los parámetros + getters todas las propiedades + setters de <i>nombre</i> y <i>precio</i> + toString() devuelve texto

La clase Articulo representará un artículo de una tienda online internacional. El método toString() devolverá una cadena con el formato:

Artículo código / nombre / precio

Crea una clase de pruebas con un método *main* que realice las siguientes acciones:

- Crea un artículo con valores: 1, "Auriculares SONY FW22", 30€
- Crea otro artículo con valores: 2, "Teclado Mehosy RT302", 50€
- Crea otro artículo con valores: 3, "Ratón inalámbrico Delta 3", 25€
- A continuación, imprime en pantalla los tres artículos.
- Súbele el precio al primer artículo a 39.00€ y vuélvelo a imprimir.
- Cambia el precio del segundo artículo a -50€ y vuélvelo a imprimir.

6. Observa la clase del diagrama UML.

Escríbela y en el método *haceRuido* simplemente imprime en pantalla “No sé qué ruido hago. Puedo ser muchas cosas”

A continuación, extiende la clase anterior creando dos subclases llamadas *Felino* y *Canido*.

Extiende la clase *Felino*, creando dos subclases llamadas *Gato* y *Tigre*.

Extiende la clase *Canido*, creando la clase *Perro*.

Sobrescribe el método *haceRuido* en las siguientes clases para que se comporte del siguiente modo:

- En la clase *Gato* debe imprimirse “Miauuuu”
- En la clase *Tigre* debe imprimirse “Roaaaaaooaarr”
- En la clase *Perro* debe imprimirse “Guau”

Por último, escribe una clase *PruebaMamiferos* que cree un objeto de cada una de las clases creadas y pruebe su método *haceRuido*.

OJO: en este caso **NO** nos interesa que los métodos *haceRuido* llamen al método de su clase padre con *super.haceRuido()*;

Mamifero
- nombre: texto
- anioNacimiento: entero
+ getters y setters
+ haceRuido()

7. Realiza un diagrama UML en el que se modele una herencia entre clases de dispositivos de un ordenador sabiendo que:

- Todos los dispositivos se pueden encender/apagar y saber si están encendidos o no.
- Todos los dispositivos tienen una marca y un modelo y unos métodos que permiten acceder y modificar estos valores.
- El procesador es un dispositivo que puede sumar, restar, multiplicar y dividir dos números enteros, devolviendo el resultado correspondiente. Además, llevará la cuenta de cuántas operaciones ha realizado desde que se encendió por última vez.
- La memoria RAM es un dispositivo capaz de guardar hasta 100 números enteros. Además, permite acceder a cada uno de estos números según su posición (0...99) para modificar su valor (escribir en memoria) o bien obtener dicho valor (leer de memoria). Además, llevará la cuenta de cuántas operaciones de lectura y cuántas operaciones de escritura se han realizado desde que se encendió por última vez.
- La tarjeta de vídeo es un dispositivo capaz de mostrar en la pantalla números enteros y textos. Además, llevará la cuenta de cuántas operaciones de mostrar un número entero y cuántas operaciones de mostrar un texto se han realizado desde que se encendió por última vez.

Una vez tengas el diagrama con la jerarquía de clases y las propiedades/métodos que componen cada clase, codifícalo.

Por último, haz una clase *PruebaDispositivos* que cree un procesador, una memoria y una tarjeta de vídeo y pruebe cada uno de los métodos que has desarrollado.

8. Retoma el ejercicio anterior y sobrescribe el método `toString()` en todas las clases, teniendo en cuenta que las subclases deben hacer uso del método `toString` de su superclase y, además, imprimir el estado de sus propiedades. Por ejemplo, esta podría ser la cadena de texto devuelta por el método *toString()* de un objeto de la clase `Procesador`.

Dispositivo: marca=Intel, modelo=i5, encendido=true. Procesador: operaciones=6.

9. Retoma el ejercicio anterior y añade el siguiente método abstracto a la clase `Dispositivo`:

`public abstract void resetContadores();`

Este método deberá ser implementado por las subclases y su comportamiento consiste en poner a cero todos los contadores que haya en la subclase.

10. Imagina que ahora decidimos reorganizar clase `Dispositivo` añadiendo las siguientes subclases:

- `Entrada`: que representaría a los dispositivos de solo entrada de datos en un ordenador (teclado, ratón...)
- `Salida`: que representaría a los dispositivos de solo salida de datos en un ordenador (altavoces, pantalla...)
- `EntradaSalida`: que representaría a los dispositivos que permiten tanto la entrada como la salida de datos en un ordenador (impresora...)

Supongamos que estas subclases no tienen métodos propios, escribe cómo declararías cada una de las tres clases. A continuación, responde a las siguientes preguntas:

- ¿Tendría sentido crear un objeto de la clase `Entrada` con un `new Entrada()`?
- Ahora queremos crear una clase `Impresora` extendiéndola desde la clase `EntradaSalida` ¿qué métodos heredaría con código? ¿Y cuáles sin código?
- ¿Se podría crear la clase `Impresora` anterior, pero extendiendo de las clases `Entrada` y `Salida` simultáneamente?
- ¿Tendría sentido decir que la clase `Entrada` es abstracta y final a la vez? Explícalo razonadamente.

11. Retoma el código del ejercicio 7 y añade el siguiente constructor a la clase `Dispositivo`:

`public Dispositivo(String marca, String modelo)`

Este constructor debe asignar la marca y el modelo a las propiedades respectiva. Sin añadir ningún constructor más a la clase `Dispositivo`, toma las medidas que creas oportunas para arreglar los errores que se producen en las subclases.

12. Partimos de nuevo del código del ejercicio anterior para realizar una nueva clase llamada *TestDispositivos* que se dedica a encender y apagar dispositivos 100 veces para comprobar su correcto funcionamiento.

Esta clase contendrá un método *main* que utilizará un array de 5 dispositivos de distintas clases (por ejemplo: 2 procesadores, 2 memorias y 1 tarjeta de vídeo). Debes escribir el código que permite encender y apagar 100 veces los 5 dispositivos del array.

OJO: no vale hacer 5 bucles, uno por dispositivo... Esta solución no sería “escalable” ¿Qué pasa si mañana el array es de 50 dispositivos?

13. Observa las siguientes interfaces:

```
public interface IArticulo {  
    String getNombre();  
    void setNombre(String nombre);  
    double getPrecio();  
    void setPrecio(double precio);
```

```
    // Sobrescribimos el método toString de Object para que  
    // devuelva una cadena con el formato:  
    // Artículo: nombre. Precio: precio €  
    @Override
```

```
    String toString();  
}
```

Fichero
IArticulo.java

```
public interface ICarroCompras {
```

```
    // Añade un artículo más al carro, añadiéndolo a la  
    // colección (array) existente.  
    void meteEnCarro(IArticulo art);
```

```
    // Recorre el carro mostrando los artículos almacenados  
    void muestraArticulosCarro();
```

```
    // Devuelve el número de artículos guardados en el carro  
    int getNumArticulosActual();
```

```
    // Elimina los artículos del carro  
    void vaciarCarro();
```

```
    // Devuelve el sumatorio de los precios de los artículos  
    // guardados en el carro  
    double calculaImporteCarro();  
}
```

Fichero
ICarroCompras.
java

Imagina que te doy estos dos ficheros y te pido que crees una clase *Articulo* que implemente la interfaz *IArticulo* y también la clase *CarroCompras* que implemente la interfaz *ICarroCompras*.

Fíjate en la potencia expresiva que tienen las interfaces. Con solo dos ficheritos .java que el diseñador puede facilitar a su equipo de desarrolladores se puede expresar lo siguiente:

- Quiero un diseño con, al menos, 2 clases.
- Quiero que cada clase tenga al menos el conjunto de métodos que especifica la interfaz.
- Quiero que los métodos se llamen exactamente así y tengan esos parámetros de entrada y salida.
- Estoy dejando intuir las propiedades que debe tener cada clase.
- Y además también se deja entrever cómo se relacionan ambas clases.

Además, el compilador se convierte en nuestro aliado porque dará error si alguna de las clases no cumple correctamente con la interfaz que implementa.

NOTA: La finalidad de este ejercicio es la reflexión sobre estos conceptos, no hay que desarrollar código alguno.

14. Dada la siguiente interfaz:

```
public interface InterfazDocumento {  
    String getTitulo();  
    void setTitulo(String titulo);  
    String getContenido();  
    void setContenido(String contenido);  
    int getNumPaginasImpresion();  
    void setNumPaginasImpresion(int numPaginas);  
}
```

Crea una clase *DocumentoTexto* que implemente *InterfazDocumento*. A continuación, crea una clase *PruebaDocumentoTexto* que tenga un método *main* que cree un objeto de la clase que has creado que pruebe todos los métodos que has creado.

15. Continúa el ejercicio anterior, añadiendo la siguiente interfaz:

```
public interface InterfazImpresora {  
    String getMarca();  
    String getModelo();  
    void encender();  
    void apagar();  
    void cargarBandeja(int numFolios);  
    void imprimir(InterfazDocumento iDocum);  
    String getEstado();  
}
```

Crea una clase *Impresora* que implemente la *InterfazImpresora* teniendo en cuenta que:

- El método *cargarBandeja* añade el número de folios que se reciben como parámetro a los que hubiera en la bandeja.
- El método *getEstado* devolverá un texto que describa el estado en el que está la impresora actualmente.
- Si la impresora se enciende el texto del estado de la impresora deberá establecerse a "OK". En el caso de que se apague se deberá poner a "Fuera de servicio".
- Cuando se llame al método *imprimir* se debe imprimir en pantalla el siguiente mensaje: "Se está imprimiendo el documento *título* que ocupa xx páginas", siempre que la impresora esté encendida y haya suficientes folios para imprimir el documento.
- Sin embargo, en el caso de que el número de folios en la bandeja no sea suficiente para imprimir un documento completo entonces se emitirá el mensaje: "Error: no hay suficiente papel para imprimir el documento *título*" y se modificará el estado de la impresora estableciendo el texto "Falta papel".
- Si la impresora estaba apagada y se intenta imprimir se debe mostrar un mensaje error.

A continuación, crea una clase *PruebaImpresora* que tenga un método *main* que cree un objeto de tipo *Impresora* que tenga como marca "HP" y modelo "Laserjet 2000". Se debe cargar con 5 folios. Por otro lado, debes crear dos documentos, uno que ocupe 4 folios y otro que ocupe 3. Debes intentar imprimir ambos documentos y ofrecer al usuario los mensajes que correspondan.

16. Dada la siguiente interfaz:

```
public interface InterfazFichero {  
    String getNombre();  
    void setNombre(String nombre);  
    void setCodificacion(String codificacion);  
    String getCodificacion();  
    void abrir();  
    void cerrar();  
    void borraContenido();  
    void agregaContenido(String contenido);  
    String getContenido();  
    int getTamañoEnBytes();  
}
```

Crea una clase que implemente *InterfazFichero* sabiendo que:

- Los ficheros deben crearse con un nombre antes de poder ser abiertos.
- Para poder manipular un fichero debe estar abierto.

- El método *agregaContenido* añade el nuevo contenido concatenándolo al final del contenido que hubiera previamente.
- La codificación del fichero puede ser “UTF-8” o “UTF-16”. Se aplicará “UTF-8” como codificación por defecto y también, en el caso de que el usuario especifique alguna otra codificación desconocida.
- El método *getTamanoEnBytes* devolverá el número de bytes que ocupa el contenido del fichero sabiendo que cada carácter en UTF-8 ocupa 1 byte y que cada carácter en UTF-16 ocupa 2 bytes.

A continuación, crea una clase *PruebaFichero* que tenga un método *main* que cree un objeto de la clase que has creado que pruebe todos los métodos que has creado.

17. Dada la siguiente interfaz:

```
public interface InterfazMensajeMail {  
    void setAsunto(String asunto);  
    String getAsunto();  
    void setContenido(String contenido);  
    String getContenido();  
    void setRemitente(String direccionEmail);  
    String getRemitente();  
    void setDestinatario(String direccionEmail);  
    String getDestinatario();  
    void setFicheroAdjunto(InterfazFichero fichero);  
    void eliminaFicheroAdjunto();  
    InterfazFichero getFicheroAdjunto();  
    void envia();  
}
```

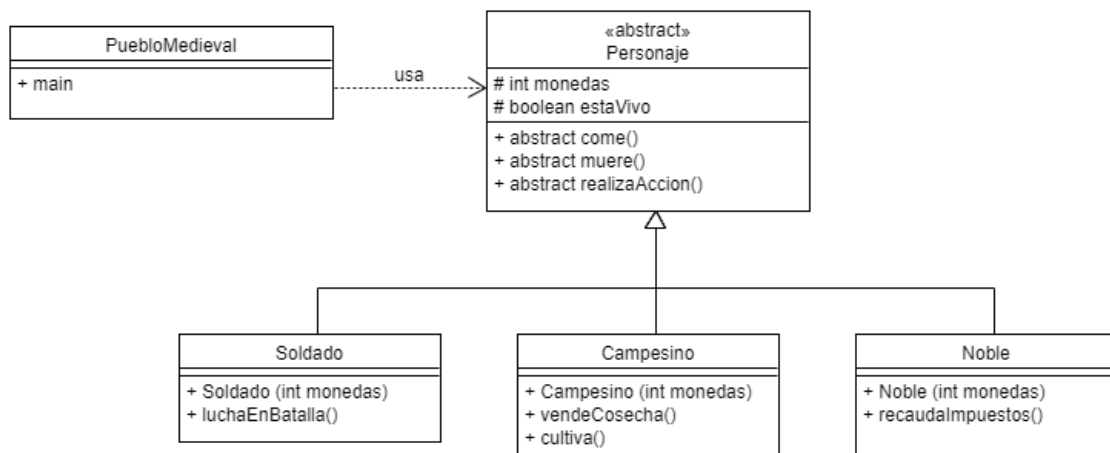
Crea una clase que implemente *InterfazMensajeMail* sabiendo que:

- Los métodos *setRemitente* y *setDestinatario* deben comprobar que el parámetro contiene el carácter @ y que la longitud de la cadena es mayor o igual a 5 caracteres. En caso contrario no se guardarán los valores en las propiedades correspondientes.
- El método *envia* debe comprobar que el existe la dirección del remitente y del destinatario, emitiendo un mensaje de error en caso contrario. Si el asunto o el contenido están vacíos entonces se imprimirá un mensaje de advertencia tipo “OJO: estás enviando un mensaje sin asunto/contenido”. A continuación, se enviará el mensaje y se imprimirá en la pantalla alguno de los siguientes mensajes según corresponda:
 - “Mensaje enviado de remitente@xxxx a destinatario@yyyy con asunto xxxxxxxxxxxx”
 - “Mensaje enviado de remitente@xxxx a destinatario@yyyy con asunto xxxxxxxxxxxx y con un fichero adjunto de xx bytes”

A continuación, crea una clase *PruebaMensajeMail* que tenga un método *main* que cree los objetos necesarios para probar todos los métodos de la clase que has creado.

18. Descarga el documento “Tabla de modificadores.docx” de la Moodle y complétalo, explicando qué hace cada modificador sobre cada elemento y o bien escribiendo “NO APLICA” si un modificador no se puede utilizar sobre cierto elemento del lenguaje.

RETO:



Codifica el diagrama UML anterior sabiendo que:

- Cuando creamos un soldado, campesino o noble tenemos que indicar el número de monedas inicial que posee y además marcarlo como "vivo".
- Cada vez que un personaje *come* gasta dinero: el soldado gasta 5 monedas, el campesino 3 y el noble 10. Después de comer se debe imprimir un mensaje en el que se muestre el tipo de personaje, las monedas que le restan y un texto de la acción realizada. Ejemplo: "Soldado COME – Tiene X monedas"
- Cada vez que un personaje *muere* se debe cambiar su correspondiente propiedad y se emite un mensaje del tipo: "Soldado MUERE – Tiene X monedas"
- Los soldados, además, pueden luchar en batallas y ganar 50 monedas en cada contienda. Además, se imprimiría: "Soldado LUCHA – Tiene X monedas"
- Los campesinos, pueden vender su cosecha y ganar 50 monedas, en este caso se imprime "Campesino VENDE COSECHA – Tiene X monedas ". También pueden cultivarla imprimiéndose el mensaje "Campesino CULTIVA – Tiene X monedas".
- Los nobles pueden recaudar impuestos y ganar 500 monedas cada vez, imprimiéndose el mensaje "Noble RECAUDA – Tiene X monedas".
- Todos los personajes deben implementar el método *realizaAccion* que realizará lo siguiente:
 - Si el personaje está muerto se imprime un mensaje del tipo: "Soldado muerto – SIN ACCIÓN".
 - Si no está muerto entonces se genera un número aleatorio entero entre 0 y 100, ambos inclusive y se aplican las acciones que correspondan siguiendo lo siguiente:
 - Para los soldados: entre 0 y 20 mueren, entre 21 y 60 luchan, para otros valores comen.

UD 4 – Relaciones entre clases

- Para los campesinos: entre 0 y 10 mueren, entre 11 y 60 cultivan, entre 61 y 70 venden su cosecha y para otros valores comen.
- Para los nobles: entre 0 y 5 mueren, entre 6 y 20 recaudan impuestos y para el resto de los valores comen.
- La clase *PuebloMedieval* tendrá un *main* que cree un array de 5 personajes, lo rellenará con 1 noble, 2 campesinos y 2 soldados y le preguntará al usuario ¿cuántos ciclos de acciones desea visualizar? Entendemos que un ciclo de acciones es un bucle capaz de recorrer todo el array llamando al método *realizaAccion* de cada personaje.
- Observa el ejemplo de ejecución para 4 ciclos, sabiendo que la situación inicial de partida es:
 - Se crean 2 soldados, uno con 20 monedas y el otro con 35.
 - Se crean 2 campesinos, uno con 11 monedas y otro con 15.
 - Se crea 1 noble con 210 monedas.

CICLO 1:

Soldado COME - Tiene 15 monedas

Soldado COME – Tiene 30 monedas

Campesino COME – Tiene 8 monedas

Campesino CULTIVA - Tiene 15 monedas

Noble COME - Tiene 200 monedas

CICLO 2:

Soldado COME – Tiene 10 monedas

Soldado LUCHA – Tiene 80 monedas

Campesino CULTIVA – Tiene 8 monedas

Campesino VENDE COSECHA – Tiene 65 monedas

Noble COME - Tiene 190 monedas

CICLO 3:

Soldado MUERE - 10 monedas

Soldado COME - 75 monedas

Campesino CULTIVA - 8 monedas

Campesino COME - 62 monedas

Noble COME - 180 monedas

CICLO 4:

Soldado muerto – SIN ACCION

Soldado COME – Tiene 70 monedas

Campesino VENDE COSECHA - Tiene 58 monedas

Campesino COME – Tiene 59 monedas

Noble RECAUDA – Tiene 680 monedas